翁恺课 · 第 4 主题

W4 循环

对齐翁恺《程序设计入门——C语言》第 4 讲 · 目标：会用 while/do-while/for 让程序"反复做事"、会用 break/continue 控制循环、会写嵌套循环 · 不覆盖：数组与函数（W5/W6）

4.1 为什么要循环：程序也会"反复做同一件事"

到目前为止，每段代码都只执行一次。但真实程序经常要**重复**：把 1 加到 100、连续读 10 个数、输错密码重来。翁恺的比喻：**循环 = 给程序装一台"复读机"**，一句话反复念，直到你喊停。更贴切的说法：循环像**流水线上的工人**——同一道工序反复做，做完一件检查一次，不合格继续，合格才放行。

> **一句话：**循环让**一段代码反复执行**，直到**条件不成立**才停下来。C 里三种循环：while、do-while、for。

4.2 while：先判断，后执行

while 的流程是：**先看条件**，成立才进循环体，跑完回来**再看条件**……直到条件不成立。注意——如果第一次条件就不成立，循环体**一次都不会执行**。经典例子：求 1+2+…+n：

```c
/* wk4_sum.c — while 求和 1+2+...+n（gcc 实跑）*/
#include <stdio.h>

int main(void)
{
    int n, i = 1, sum = 0;
    printf("求和到几？");
    scanf("%d", &n);

    while (i <= n) {
        sum += i;
        i++;
    }
    printf("1+2+...+%d = %d\n", n, sum);
    return 0;
}
```

输入 10，实跑输出：

```
求和到几？1+2+...+10 = 55
```

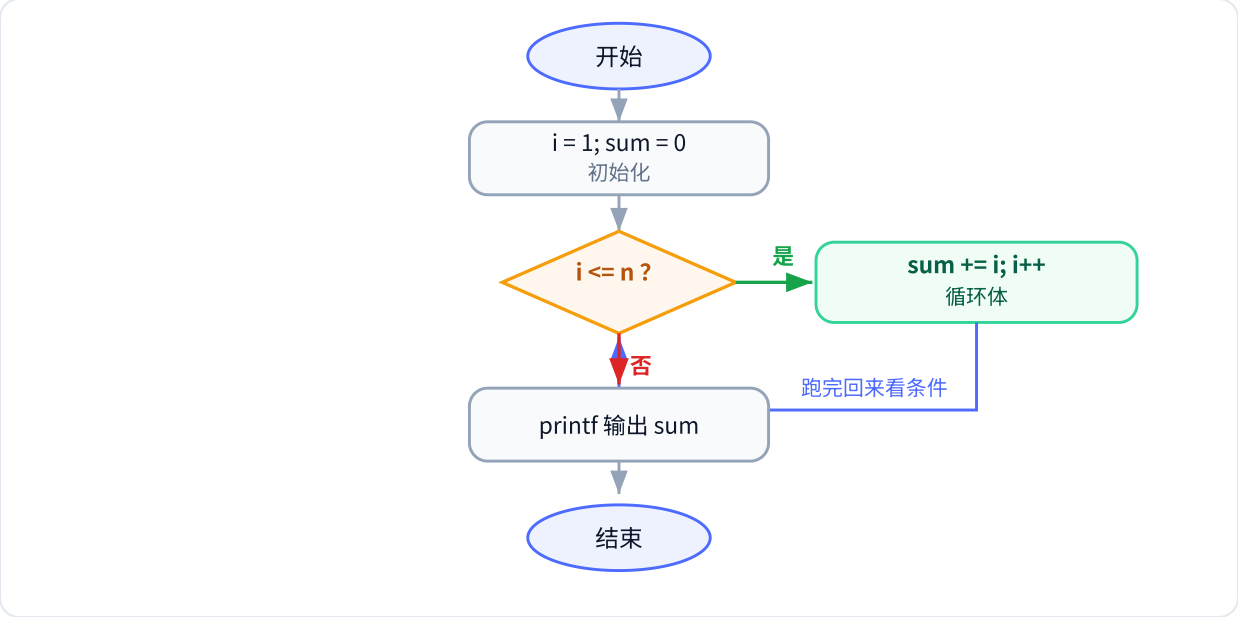


图 W4-2　while 求和流程：初始化 → 判断 i≤n（是→执行循环体 sum+=i; i++，再回来判断；否→输出结果结束）——"先判断、后执行"

> **新手坑：忘了更新循环变量 = 死循环。**如果上面代码漏写 `i++;`，`i` 永远是 1，条件 `i <= n` 永远成立——程序**卡死**。gcc 编译 0 警告 0 错误（**编译期看不见**），运行才暴露：屏幕一直滚，只能 Ctrl+C 终止。循环体里一定要有"让条件趋向不成立"的一步。

4.3 do-while：先做一次，再判断

`do-while` 反过来：**先执行一次循环体，再判断条件**。所以它**至少执行一次**。最适合"无论如何都要先问一遍"的场景——比如输密码：总得先让你输一次，错了才重来：

```
/* wk4_dowhile.c — do-while 输入校验（gcc 实跑）*/
#include <stdio.h>

int main(void)
{
    int n;
    do {
        printf("输入一个 1~100 的数：");
        scanf("%d", &n);
    } while (n < 1 || n > 100);

    printf("收到：%d\n", n);
    return 0;
}
```

先输入 `200`（非法），再输入 `42`，实跑输出：

```
输入一个 1~100 的数：输入一个 1~100 的数：收到：42
```

> **翁恺提醒：**do-while 的**分号**别忘——`} while（条件）;` 结尾的分号是语句的一部分，漏掉会编译报错（见 4.7 坑②）。while 和 do-while 怎么选？**必须至少做一次 → do-while；可能一次都不做 → while。**

4.4 for：计数循环的"三件套"

for 把循环的三件事写在一行：**初始化、条件、更新**。翁恺的说法：这是"三件套"一次配齐，适合**明确知道要循环几次**的场景：

```c
/* wk4_for.c — for 计数循环 1..5（gcc 实跑）*/
#include <stdio.h>

int main(void)
{
    int i;
    for (i = 1; i <= 5; i++) {
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

实跑输出：

```
1 2 3 4 5
```

for 三件套	做什么
`i = 1`	初始化：进入循环前执行一次，只做一次（这里从 1 开始数）
`i <= 5`	条件：每次进循环前检查，成立才执行循环体
`i++`	更新：每次执行完循环体后做，让 i 朝"不成立"方向走



图 W4-1　三种循环概念图：骨架与特点对比。已知次数用 for；至少做一次用 do-while；其余用 while（三种都能互相改写）

4.5 break / continue：急刹车与跳过

循环中途想变卦？两个"控制开关"：`break` = **急刹车**，整个循环立刻退出；`continue` = **跳过这次**，不执行剩下的循环体，直接进入下一次。看例子：

```
/* wk4_bc.c — break / continue 行为演示（gcc 实跑）*/
#include <stdio.h>

int main(void)
{
    int i;
    for (i = 1; i <= 10; i++) {
        if (i == 3) continue;   /* 跳过数字 3 */
        if (i > 6) break;       /* i 超过 6 就整个退出 */
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

实跑输出：

```
1 2 4 5 6
```

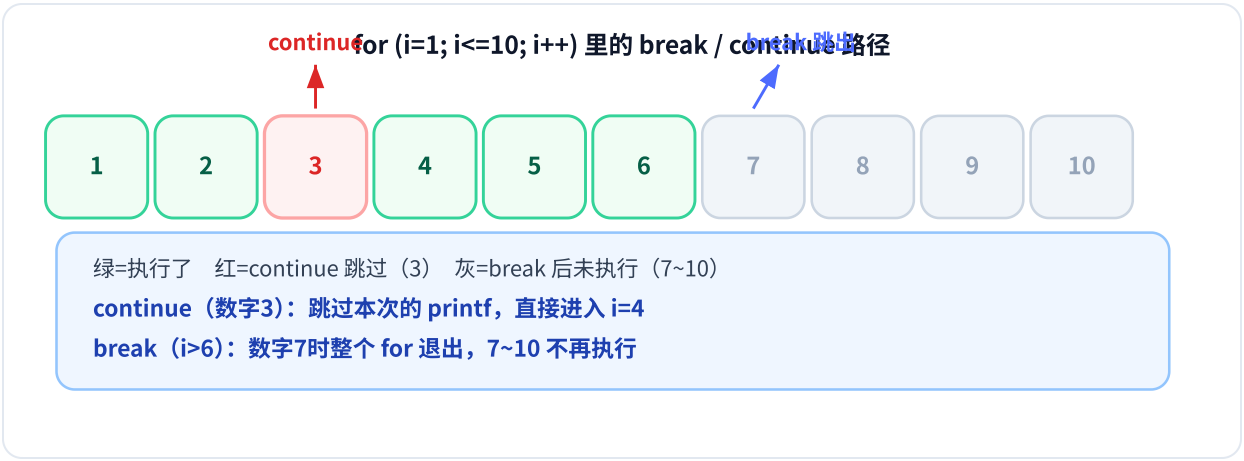


图 W4-3 break / continue 路径：continue 只跳过"本次"（3 不打印），break 直接"跳出整个循环"（7 开始不再执行），实测输出 1 2 4 5 6

> **新手坑：break 只跳出"最近的一层"循环。**嵌套循环里写 break，跳出的是内层，外层照常继续。想一次跳出多层？要么加"退出标志"，要么用 `goto`（初学者先别用）。

4.6 嵌套循环：循环里面套循环

循环体里还能放循环，叫**嵌套循环**。经典作品——九九乘法表：外层管"行"，内层管"列"：

```
/* wk4_mul.c — 嵌套循环：打印九九乘法表（gcc 实跑）*/
#include <stdio.h>

int main(void)
{
    int i, j;
    for (i = 1; i <= 9; i++) {
        for (j = 1; j <= i; j++) {
            printf("%d*%d=%2d  ", j, i, i * j);
        }
        printf("\n");
    }
    return 0;
}
```

实跑输出（前 3 行）：

```
1*1= 1
1*2= 2  2*2= 4
1*3= 3  2*3= 6  3*3= 9
```

> **翁恺提醒：**看嵌套循环先分清楚**外层跑一圈、内层跑整轮**。上面第 3 行：外层 i=3 时，内层 j 从 1 跑到 3（j<=i），所以一行正好 3 个式子。内层条件写 `j <= i` 让表格变成"三角形"，如果写 `j <= 9` 就是完整方

> 形表。

4.7 新手坑集中营（本章最容易错的 3 件事）

1. **忘了更新循环变量 → 死循环**：条件永远成立，编译 0 警告但运行卡死。循环体里必须有"让条件趋向不成立"的一步。
2. **do-while 忘分号 → 编译错误**：`} while (条件)` 结尾的分号不能省。gcc 实测报：`error: expected ';' before 'printf'`（见下方诊断）。
3. **条件里把 = 当 == → 恒真死循环**：`while (i = 1)` 是"给 i 赋值 1"，值非 0 恒真。gcc 实测报警告 `[-Wparentheses]`，编译能过但程序卡死。

```
两个坑的真实 gcc 诊断（-Wall -Wextra 实测）：*/
① do-while 忘分号：
wk4_bug_semi.c:10:20: error: expected ';' before 'printf'
   10 |     } while (n > 0)        /* 忘了分号！ */
      |                    ^
      |                    ;
② while 条件里用 = 赋值：
wk4_bug_assign.c:7:12: warning: suggest parentheses around assignment
used as truth value [-Wparentheses]
    7 |     while (i = 1) {        /* 想写 i == 1？ */
      |            ^
```

4.8 一分钟自测（答案见练习册）

判断 / 填空：

① `while` 与 `do-while` 的本质区别是 ____：do-while 保证循环体至少执行 ____ 次。

② `for（初始化；条件；更新）`三部分里，进入循环前只执行一次的是 ____；每次进循环前检查的是 ____。

③ `break` 的作用是 ____；`continue` 的作用是 ____。

4.9 练习指引

去配套练习册做 W4 练习。翁恺建议本讲里程碑：**不看参考写一个"猜数字"游戏**——程序想一个 1~100 的数，你猜，它提示"大了/小了"，用 while 反复读输入，用 break 在猜中时退出，最后统计猜了几次。

本讲义为自写原创讲解（对齐翁恺《程序设计入门——C语言》第 4 讲），仅作课堂伴读；示例代码均用 `gcc -Wall -Wextra` 实测通过（MinGW-W64 gcc 16.1.0），输出为真实运行结果（含 gcc 警告原文）。视频与图片版权归原作者所有，本讲义不含课程原素材。